Отношения элементов

В объектно-ориентированном проектировании существуют два фундаментальных способа построения новых классов на основе имеющихся: **наследование** и **композиция**. Если наследование выражает отношение “является” (is-a), то композиция и смежные с ней связи описывают отношения “содержит как часть” (has-a) или “использует” (uses-a).

При проектировании архитектуры принято выделять три ключевых вида связей между объектами: **ассоциацию**, **агрегацию** и **композицию**. В Python разница между ними заключается не в синтаксических ключевых словах, а в логике владения и управлении жизненным циклом объектов

Ассоциация

> **Ассоциация** – это наиболее слабая форма связи, при которой объекты работают вместе, но остаются полностью независимыми. Один объект запрашивает услугу или вызывается у другого объекта (оказывает услугу)

- Связь по жизненному циклу: Нет связи. Объекты создаются и уничтожаются абсолютно независимо.
- Пример: Владелец и собака. Владелец может выгулять собаку, но собака и владелец существуют независимо друг от друга[10].

```python
class Dog:
    def __init__(self, name: str):
        self.name = name

class Owner:
    def __init__(self, name: str):
        self.name = name
    def walk_dog(self, dog: Dog) -> None:
        print(f"{self.name} выгуливает собаку по кличке {dog.name}")
```

Объекты создаются независимо друг от друга:

```python
owner = Owner("Алексей")
dog = Dog("Шарик")
```

А взаимодействуют друг с другом через передачу объекта в метод

```python
owner.walk_dog(dog)
```

```
Алексей выгуливает собаку по кличке Шарик
```

Агрегация

> **Агрегация** представляет собой отношение “целое — часть”, при котором сложный объект состоит из других независимых объектов. Составные элементы создаются снаружи и передаются в сложный объект.

- Связь по жизненному циклу: Слабая. Если «целое» будет уничтожено или удалено из системы, его составные «части» продолжат жить и могут использоваться в других местах.

- Пример: Автомобиль и его отдельные атрибуты/аксессуары (например, внешняя аудиосистема или съёмный навигатор), либо сотрудник и отдел. Если сотрудник увольняется из отдела, объект сотрудника продолжает существовать в системе

```python
class Wheel:
    def __init__(self, size: int):
        self.size = size

class Car:
    def __init__(self, model: str, wheels: list[Wheel]):
        self.model = model
        self.wheels = wheels
```

Части создаются отдельно:

```python
w1, w2, w3, w4 = Wheel(17), Wheel(17), Wheel(17), Wheel(17)
car = Car("Sedan", [w1, w2, w3, w4])
```

Если объект car будет уничтожен, то колеса остаются

```python
del car
w1.size
```

```
17
```

Композиция

> **Строгая композиция** — это отношение “целое — часть”, при котором составные элементы не могут существовать без главного объекта. Главный объект полностью управляет созданием и жизненным циклом своих частей.

- Связь по жизненному циклу: Строгая. Части создаются внутри контейнера. При уничтожении контейнера автоматический сборщик мусора Python уничтожает и все вложенные объекты.
- Пример из жизни: Двигатель и его цилиндры, или дом и его комнаты. Комната не имеет смысла вне контекста конкретного здания.

```python
class Room:
    def __init__(self, area: float):
        self.area = area

class House:
    def __init__(self):
        # Комнаты создаются непосредственно внутри конструктора дома
        self.rooms = [Room(20.0), Room(15.0)]
    def get_total_area(self) -> float:
        return sum(room.area for room in self.rooms)

house = House()
house.get_total_area()
```

```
35.0
```

При удалении house комнаты за пределами объекта недоступны и будут удалены

```python
del house
```

Особенности реализации в Python

В отличие от некоторых языков со статической типизацией, в Python все эти связи организуются через хранение ссылок на другие объекты или списки ссылок в атрибутах экземпляра.

Использование агрегации и ассоциации вместо жесткой композиции снижает степень связанности кода, что делает программы более гибкими и удобными для тестирования.

Управление памятью: В Python жизненный цикл при композиции регулируется подсчетом ссылок и сборщиком мусора. Когда уничтожается родительский объект, ссылки на вложенные объекты обнуляются, и они удаляются из памяти.